

XSIAM Deep Dive · Hoja del alumno

Todo lo que haces con scripts ocurre el **Día 2**, en tu propia máquina. Ven con **Python 3.9+** y **Git** ya instalados. Del Día 3 en adelante trabajas solo dentro de XSIAM.

DÍA 2 · LOS SEIS PASOS

1. **Clona el repositorio** del curso e instala la dependencia. Necesitas **Python 3.9 o superior con pip**.

```
git clone <URL-que-te-dio-el-instructor>
cd xsiam-lab/simulator
pip install -r requirements.txt
```

Todos los comandos se corren desde `simulator/`. En Windows escribe `py` en vez de `python`, y `py -m pip` en vez de `pip`.

2. **Crea tus 4 colectores HTTP** en XSIAM. Settings → Data Sources → Add Data Source → Custom Integrations → HTTP Collector

Nombre	Vendor	Product	Log Format	Compresión
EQn-waf	imperva	securesphere	CEF	none
EQn-fw	andina	ngfw	Raw	none
EQn-edr	andina	edr	JSON	none
EQn-auth	andina	idp	JSON	none

De cada colector anota **dos cosas**: la **API key** y el **nombre del dataset** que te muestra el tenant. Ese nombre lo vas a usar los tres días siguientes.

```
waf → _____ fw → _____
edr → _____ auth → _____
```

3. **Configura**.

```
cp config.example.json config.json
```

Abre `config.json` y pega la URL del tenant y tus 4 API keys.

4. **Comprueba que llegán**.

```
python run_lab.py check
```

Deben salir **4 OK**. Verifícalo también en el Query Builder:

```
dataset = <tu_dataset>
| filter _raw_json contains "xsiam-deep-dive"
| limit 10
```

5. **Carga 48 horas de tráfico normal**. Tarda un par de minutos.

```
python run_lab.py baseline --hours 48 --eps-scale 0.4
```

Mientras corre, ten abierto **Data Management → Data Ingestion** y mira subir las barras.

6. **Entrégale tus 4 API keys al instructor**.

 Sin esto, el viernes tu SOC **no recibe el ataque** y te quedas sin capstone. No es una amenaza, es cómo funciona: él lanza la campaña contra tus colectores.

DÍAS 3, 4 Y 5

No corres ningún script más. Todo tu trabajo pasa **dentro de XSIAM**: parsers, reglas de correlación, feed de indicadores y playbooks.

Los datos de ataque te los manda el instructor sin avisar. Si intentas `python run_lab.py attack` te saldrá un mensaje diciendo que el escenario lo controla él. Es **a propósito**: el viernes tiene que ser una sorpresa.

LOS TRES COMANDOS QUE USARÁS

Comando	Para qué
<code>python run_lab.py check</code>	¿Mis 4 colectores reciben eventos?
<code>python run_lab.py baseline --hours 48 --eps-scale 0.4</code>	Llenar mi tenant con tráfico normal del negocio
<code>python run_lab.py baseline --hours 2 --eps-scale 0.5</code>	Un chorro corto, para ver los datos llegar en vivo

Agrega `--dry-run` a cualquiera y en vez de enviar escribe los eventos en archivos dentro de `out/`. Sirve para ver exactamente qué formato tiene cada fuente antes de escribir el parser.

SI ALGO FALLA

Qué ves	Qué pasa
No module named requests	Te faltó el paso 1: <code>pip install -r requirements.txt</code>
No module named pip	<code>py -m ensurepip --upgrade</code> . Si sigue, reinstala Python desde python.org marcando <i>Add python.exe to PATH</i>
HTTP 401	La API key está mal copiada (casi siempre sobra un espacio) o el colector está deshabilitado
HTTP 404	La URL del tenant está mal escrita en <code>config.json</code>
Sale 200 pero no veo nada en XSIAM	Espera 1-2 minutos y comprueba que estás mirando el dataset correcto
Ningún colector configurado	Te faltó <code>cp config.example.json config.json</code>
No sale nada de tu máquina	La red bloquea la salida hacia el tenant. Prueba con otra conexión o comparte datos del móvil
<code>python: command not found</code>	En Windows suele ser <code>py</code> en vez de <code>python</code>

No subas tu `config.json`: tiene tus API keys y ya está en el `.gitignore`. · **Dato útil para el Día 3**: las cuatro fuentes traen formatos distintos a propósito. El WAF llega en CEF sobre syslog, el firewall en texto plano con campos separados por `|`, y el EDR y la autenticación en JSON anidado. El texto crudo queda en `_raw_log` para las dos primeras y en `_raw_json` para las otras dos. Míralos con `--dry-run` antes de escribir un solo parser.